

Computer Science & Software Engineering Knowledge Base

A curated knowledge base for a retrieval-augmented generation (RAG) chatbot. It covers core programming, data structures, databases, web development, machine learning, and software engineering concepts. The chatbot should use retrieved sections from this document as its primary factual context.

Coverage: Python, JavaScript, TypeScript, SQL, data structures and algorithms, object-oriented programming, React, Node.js, REST APIs, databases, Git, machine learning, and RAG fundamentals.

1. Python

1.1 Python Basics

Python is a high-level, general-purpose programming language known for readable syntax and a large standard library. A Python program is commonly organized into modules and packages.

- Variables are names bound to objects; Python is dynamically typed, so a variable does not need a declared type.
- Common built-in types include int, float, str, bool, list, tuple, set, dict, and NoneType.
- A list is ordered and mutable. A tuple is ordered and immutable. A set stores unique elements. A dictionary stores key-value pairs.
- Indentation defines code blocks in Python.

1.2 Functions

A function groups reusable behavior. Functions are defined with `def` and may accept positional arguments, keyword arguments, default values, and variable-length arguments.

```
def add(a, b):  
    return a + b  
  
result = add(2, 3)
```

1.3 Object-Oriented Programming in Python

A class defines the structure and behavior of objects. An object is an instance of a class. Encapsulation groups data and methods that operate on that data. Inheritance allows a class to derive behavior from another class. Polymorphism allows the same interface to be used with different implementations.

```
class Car:  
    def __init__(self, color):  
        self.color = color  
  
    def describe(self):  
        return f"Car color: {self.color}"  
  
car = Car("blue")  
print(car.describe())
```

1.4 Exceptions

Exceptions represent errors or unusual conditions during execution. `try/except` can handle exceptions; `finally` can run cleanup code regardless of whether an exception occurred.

```
try:  
    value = int("10")
```

```
except ValueError:
    print("Invalid number")
finally:
    print("Finished")
```

1.5 Comprehensions

List, set, and dictionary comprehensions provide concise ways to construct collections from iterable data.

```
squares = [x * x for x in range(5)]
even = [x for x in range(10) if x % 2 == 0]
```

2. JavaScript & TypeScript

2.1 JavaScript Fundamentals

JavaScript is a programming language widely used in web browsers and server-side environments such as Node.js. `let` and `const` are block-scoped declarations; `const` prevents reassignment of the binding.

JavaScript values include strings, numbers, booleans, objects, arrays, null, undefined, bigint, and symbols. Functions are first-class values.

2.2 Asynchronous JavaScript

JavaScript commonly handles asynchronous operations with Promises and `async/await`. A Promise represents the eventual completion or failure of an asynchronous operation.

```
async function getData() {
    const response = await fetch("/api/data");
    const data = await response.json();
    return data;
}
```

2.3 TypeScript

TypeScript is a typed superset of JavaScript that adds static type checking and compiles to JavaScript. Interfaces and type aliases can describe object shapes and function contracts.

```
interface User {
    id: number;
    name: string;
}

function greet(user: User): string {
    return `Hello ${user.name}`;
}
```

3. Data Structures & Algorithms

3.1 Arrays and Linked Lists

An array stores elements in indexed positions and supports direct access by index. A linked list stores nodes connected by references. Arrays are generally strong for indexed access; linked lists can make insertion or deletion efficient when the relevant node position is already known.

3.2 Stack and Queue

A stack follows LIFO (last in, first out). Typical operations are `push` and `pop`. A queue follows FIFO (first in, first out). Typical operations are `enqueue` and `dequeue`.

3.3 Hash Tables

A hash table maps keys to values using a hash function. Average-case lookup, insertion, and deletion are commonly $O(1)$, although collisions can affect performance.

3.4 Trees

A tree is a hierarchical data structure. In a binary tree, each node has at most two children. A binary search tree maintains an ordering where values in the left subtree are smaller and values in the right subtree are larger, subject to the chosen duplicate policy.

3.5 Graphs

A graph consists of vertices and edges. Graphs may be directed or undirected and may be weighted or unweighted. Common representations are adjacency lists and adjacency matrices.

3.6 BFS and DFS

Breadth-first search explores a graph level by level and commonly uses a queue. Depth-first search explores as far as possible along a branch before backtracking and commonly uses recursion or a stack.

3.7 Sorting

Common sorting algorithms include bubble sort, insertion sort, merge sort, quicksort, and heap sort. Merge sort has $O(n \log n)$ time complexity in its standard implementation and requires additional memory for merging. Quicksort has average $O(n \log n)$ time and worst-case $O(n^2)$ with common pivot strategies.

3.8 Big-O

Big-O notation describes how resource usage grows as input size increases. Common complexities include $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, and $O(2^n)$.

4. SQL & Relational Databases

4.1 Relational Database Concepts

A relational database stores data in tables consisting of rows and columns. A primary key identifies rows. A foreign key represents a relationship to a key in another table.

4.2 SELECT, WHERE, ORDER BY

```
SELECT name, salary
FROM employees
WHERE salary > 50000
ORDER BY salary DESC;
```

4.3 JOINS

INNER JOIN returns matching rows from both tables. LEFT JOIN returns all rows from the left table and matching rows from the right table, with NULLs where no match exists. RIGHT JOIN is the reverse. FULL OUTER JOIN returns rows from both sides where supported.

```
SELECT e.name, d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.id;
```

4.4 GROUP BY and HAVING

GROUP BY groups rows for aggregate calculations such as COUNT, SUM, AVG, MIN, and MAX. HAVING filters grouped results, while WHERE filters rows before grouping.

```
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

4.5 Indexes

An index is a data structure that can speed up lookups and sorting for selected columns. Indexes consume storage and can increase the cost of inserts, updates, and deletes because index entries may also need maintenance.

4.6 Transactions and ACID

A transaction groups database operations into a logical unit. ACID refers to atomicity, consistency, isolation, and durability.

5. React

5.1 Components

React applications are built from components. A component receives inputs called props and can maintain internal state.

5.2 Props and State

Props are inputs passed from a parent component to a child. State represents data managed by a component and can change over time.

5.3 useState

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

5.4 useEffect

useEffect is used to synchronize a component with external systems or perform side effects such as data fetching, subscriptions, or browser APIs. Its dependency array controls when the effect runs.

5.5 Context

React Context allows values to be made available to components without passing props through every intermediate component. It is useful for cross-cutting data such as theme or authenticated-user information.

6. Node.js & Express

6.1 Node.js

Node.js is a JavaScript runtime designed for executing JavaScript outside the browser. It uses an event-driven model and asynchronous I/O, which is useful for network-oriented applications.

6.2 Express

Express is a web framework commonly used with Node.js to build HTTP APIs. Routes map HTTP methods and paths to handlers. Middleware functions can inspect or modify requests and responses and can pass control to the next middleware.

```
import express from "express";

const app = express();
app.use(express.json());

app.get("/api/users", (req, res) => {
  res.json({ users: [] });
});

app.listen(3000);
```

6.3 HTTP Methods

GET retrieves data. POST commonly creates a resource or triggers an operation. PUT commonly replaces a resource. PATCH partially updates a resource. DELETE removes a resource.

7. REST APIs

7.1 REST Concepts

A REST-style API exposes resources through URLs and uses standard HTTP methods. A client and server communicate through representations such as JSON.

7.2 HTTP Status Codes

- 200 OK: request succeeded.
- 201 Created: a resource was successfully created.
- 204 No Content: request succeeded with no response body.
- 400 Bad Request: request is invalid.
- 401 Unauthorized: authentication is required or failed.
- 403 Forbidden: the server understood the request but refuses access.
- 404 Not Found: requested resource was not found.
- 500 Internal Server Error: unexpected server-side failure.

7.3 API Design

Good API design uses consistent resource naming, appropriate HTTP methods, predictable response formats, meaningful status codes, validation, authentication where required, and clear error messages.

8. MongoDB & NoSQL

8.1 MongoDB

MongoDB is a document-oriented database that stores records as BSON documents. Collections contain documents. Documents can contain nested objects and arrays.

```
{
  "name": "Alice",
  "skills": ["Python", "SQL"],
  "experience": 2
}
```

8.2 Relational vs Document Databases

Relational databases organize structured data into related tables and are strong for relational queries and transactional workloads. Document databases store flexible document structures and can be convenient when application data naturally maps to nested documents.

9. Git & Software Development

9.1 Git Basics

```
git init
git status
git add .
git commit -m "Initial commit"
git branch
git checkout -b feature/example
git pull
git push
```

A commit records a snapshot of changes. A branch provides an independent line of development. Pull retrieves changes from a remote and integrates them into the local branch according to the configured workflow.

9.2 Merge Conflicts

A merge conflict occurs when Git cannot automatically reconcile changes. The developer resolves the conflicting sections, stages the resolved files, and completes the merge or rebase according to the workflow.

10. Machine Learning

10.1 Supervised Learning

Supervised learning trains a model using labeled examples. Classification predicts discrete categories; regression predicts continuous numerical values.

10.2 Unsupervised Learning

Unsupervised learning works with data without target labels. Clustering groups similar observations; dimensionality reduction transforms data into a lower-dimensional representation.

10.3 Train, Validation, and Test Sets

A training set is used to fit model parameters. A validation set can be used for model selection and hyperparameter tuning. A test set is held out for final evaluation.

10.4 Classification Metrics

Accuracy is the fraction of predictions that are correct. Precision measures the fraction of predicted positives that are actually positive. Recall measures the fraction of actual positives that are identified. F1 is the harmonic mean of precision and recall.

```
precision = TP / (TP + FP)
recall = TP / (TP + FN)
F1 = 2 * precision * recall / (precision + recall)
```

10.5 Overfitting

Overfitting occurs when a model learns training data too specifically and performs poorly on unseen data. Common approaches to reduce overfitting include regularization, simpler models, more training data, data augmentation where appropriate, and early stopping.

11. Neural Networks & Transformers

11.1 Neural Networks

A neural network consists of layers of parameterized transformations. Training adjusts parameters to reduce a loss function, commonly using gradient-based optimization.

11.2 Attention

Attention allows a model to assign different importance to elements when producing a representation. Self-attention relates different positions within the same sequence.

11.3 Transformer

A Transformer is a neural architecture based heavily on attention mechanisms. A typical encoder-decoder Transformer uses stacked self-attention and feed-forward layers. Transformer architectures are widely used for language and sequence modeling.

12. Retrieval-Augmented Generation (RAG)

12.1 What is RAG?

Retrieval-augmented generation combines information retrieval with text generation. Instead of relying only on the language model's internal parameters, a RAG system retrieves relevant external documents and provides them as context to the model.

12.2 Typical RAG Pipeline

A common pipeline is: load documents → split documents into chunks → create embeddings → store vectors in a vector database → embed the user query → retrieve similar chunks → construct a prompt with retrieved context → generate an answer.

12.3 Embeddings

An embedding represents text as a numerical vector. Semantically similar text tends to have vectors that are close according to the chosen similarity measure.

12.4 Chunking

Chunking divides large documents into smaller sections for retrieval. Chunk size and overlap affect retrieval quality. Overlap can preserve information that crosses chunk boundaries.

12.5 Grounded Answers

A RAG assistant can be instructed to use retrieved context and state when the supplied context does not contain an answer. Source metadata such as document name and page number can be returned with the answer.

12.6 Limitations

RAG quality depends on document quality, chunking, embeddings, retrieval, context selection, and the language model. Retrieval errors can cause the model to receive incomplete or irrelevant context. A larger knowledge base does not automatically make the language model itself more capable.

13. Software Engineering Interview Quick Reference

13.1 OOP

- Encapsulation: bundling data with methods that operate on that data.

- Abstraction: exposing essential behavior while hiding implementation details.
- Inheritance: deriving one class from another.
- Polymorphism: allowing different implementations to be used through a common interface.

13.2 Common Complexity Questions

- Array indexed access: typically $O(1)$.
- Hash-table average lookup: typically $O(1)$.
- Binary search on sorted data: $O(\log n)$.
- Merge sort: $O(n \log n)$ time in the standard implementation.
- Breadth-first search: $O(V + E)$ with an adjacency-list representation.
- Depth-first search: $O(V + E)$ with an adjacency-list representation.

13.3 Debugging Process

A practical debugging process is: reproduce the problem, isolate the failing component, inspect inputs and state, form a hypothesis, test the hypothesis, apply the smallest appropriate fix, and verify the behavior with a regression test.